

Lecture 15:

Introduction to Deep Learning

ML/AI for Engineering Applications
ENV 6932

Sara Behdad
Fall 2025

Outline

- Recall: Activation Functions
- Loss Functions

The Evolution of AI and Machine Learning

Era	Focus	Example Methods	Key Limitation
1950s-1980s	Symbolic AI / Rule-Based Systems	Expert systems, decision trees	Required human-designed rules
1980s-1990s	Early Neural Networks	Perceptron, backpropagation	Limited compute and data
2000s	Machine Learning	SVMs, random forests, logistic regression	Required feature engineering
2010s-present	Deep Learning	CNNs, RNNs, Transformers	Learns features automatically

Deep learning represents the *third wave* of AI progress. It was driven by large datasets, GPU acceleration, and algorithmic breakthroughs such as backpropagation and attention mechanisms.

From Feature Engineering to Feature Learning

Traditional ML Approach

- **Manual feature design:** Domain experts extract features manually (e.g., texture, shape, color histograms).
- The model (SVM, logistic regression) only learns a mapping from features $\rightarrow$ labels.
- **Formula:**

$$y = f(w^T \phi(x))$$

where $\phi(x)$ = manually engineered feature vector.

Raw data $\rightarrow$ Handcrafted features $\rightarrow$ Model $\rightarrow$ Output

From Feature Engineering to Feature Learning

Deep Learning Approach

- The network learns **both** features and the mapping jointly:

$$y = f_{\theta}(x)$$

where θ includes parameters that define the feature extraction layers.

- **Key Advantage:** Representation learning: no need for manual feature design.

Raw data → Neural Network (learned features) → Output

What Is a Neural Network?

- A neural network is a function approximator that learns the mapping between input and output data.

where:

$$y = f_{\theta}(x)$$

- x input vector (features)
- y : output or target
- $\theta = \{W, b\}$: parameters : the weights and biases learned from data
- Each layer in a neural network performs a transformation of the data, gradually extracting more abstract features.
- Learning means finding the optimal parameters θ^* that minimize a loss function:

$$\theta^* = \arg \min_{\theta} \mathcal{L}(f_{\theta}(x), y)$$

Deep learning = hierarchical function approximation.

Universal Approximation Theorem

- A feedforward network with a single hidden layer and sufficient neurons can approximate any continuous function on a compact domain.
- This provides theoretical justification for why deep networks can represent highly complex relationships.

$$\forall f(x) \in C([0, 1]^n), \exists f_{\theta}(x) \text{ s.t. } |f(x) - f_{\theta}(x)| < \epsilon$$

The Artificial Neuron (Perceptron)

- **Mathematical Formulation**
- Each artificial neuron computes a *weighted sum* of its inputs plus a bias term:

$$z = w_1x_1 + w_2x_2 + \cdots + w_nx_n + b$$

- Then applies an **activation function** to introduce nonlinearity:

$$a = \sigma(z)$$

- where:
 - w_i : weight of input x_i
 - b : bias term (controls threshold)
 - σ : activation function (e.g., sigmoid, tanh, ReLU)

The Artificial Neuron (Perceptron)

- The neuron combines multiple inputs to produce one output.
- Without σ , the model is purely linear (limited capacity).
- The activation allows the network to model nonlinear decision boundaries.

From Neuron to Network

- Stacking neurons into *layers* forms a **neural network**.

$$a^{(l)} = \sigma(W^{(l)} a^{(l-1)} + b^{(l)})$$

- $a^{(l-1)}$: activations from previous layer
 - $W^{(l)}$: weight matrix for layer l
 - $b^{(l)}$: bias vector
 - σ : activation function applied elementwise
- **Points:**
 - Each layer transforms data into a new representation.
 - Depth = number of layers in the network.
 - Final layer gives predictions (e.g., class scores).

Forward Propagation Example

Forward propagation computes outputs layer by layer.

1. Input vector:

$$x = [x_1, x_2, x_3]$$

2. First layer computation:

$$z^{(1)} = W^{(1)}x + b^{(1)}$$

3. Apply activation:

$$a^{(1)} = \sigma(z^{(1)})$$

4. Pass to next layer, repeat:

$$\hat{y} = f_{\theta}(x)$$

Example

- Given:

$$\begin{aligned}x &= [1, 2], \\W &= [0.5, 0.2], b = 0.1, \\ \sigma &= \text{ReLU}\end{aligned}$$

- Compute:

$$\begin{aligned}z &= 0.5(1) + 0.2(2) + 0.1 = 1.0 \\a &= \max(0, 1.0) = 1.0\end{aligned}$$

Each neuron performs this operation during a forward pass. The network's prediction is obtained after all layers compute their activations.

Learning as Optimization

- **Key Idea**
- Neural networks **learn** by adjusting parameters $\theta = \{W, b\}$ to minimize a **loss function**.

$$\min_{\theta} \mathcal{L}(y, \hat{y})$$

- **Common Loss Functions**
- **Mean Squared Error (Regression)**
- **Cross-Entropy (Classification)**

$$\mathcal{L} = \frac{1}{N} \sum_i (y_i - \hat{y}_i)^2$$

$$\mathcal{L} = - \sum_i y_i \log(\hat{y}_i)$$

Training Process



1. Forward pass → compute predictions
2. Compute loss
3. Backpropagate gradients
4. Update weights

Example: Simple Linear Model

- **Model:** A single neuron as linear regression:

$$\hat{y} = wx + b$$

- Loss:

$$\mathcal{L} = \frac{1}{N} \sum (y - wx - b)^2$$

- **Gradients**

$$\frac{\partial \mathcal{L}}{\partial w} = -\frac{2}{N} \sum x(y - wx - b)$$

$$\frac{\partial \mathcal{L}}{\partial b} = -\frac{2}{N} \sum (y - wx - b)$$

- w and b are updated iteratively using these gradients.
- Over time, the line fits the data by minimizing the mean squared error.

Extending Linear Regression to Multiple Inputs (Vector Form)

- A neuron with multiple features can be written compactly as:

$$\hat{y} = W^T x + b$$

where:

- $x \in \mathbb{R}^n$: input vector
- $W \in \mathbb{R}^n$: weight vector
- $b \in \mathbb{R}$: bias scalar

- **Loss function:**

$$\mathcal{L} = \frac{1}{N} \sum_{i=1}^N (y_i - W^T x_i - b)^2$$

- **Gradient (vectorized):**

$$\nabla_W \mathcal{L} = -\frac{2}{N} X^T (y - XW - b)$$

$$\frac{\partial \mathcal{L}}{\partial b} = -\frac{2}{N} \sum_i (y_i - W^T x_i - b)$$

From Linear Model to Nonlinear Network

- To capture nonlinear relationships, we *compose* multiple linear transformations with activation functions:

$$a^{(1)} = \sigma(W^{(1)}x + b^{(1)}) \quad \text{First Layer}$$

$$\hat{y} = W^{(2)}a^{(1)} + b^{(2)} \quad \text{2nd Layer}$$

- This composition $f_{\theta}(x)$ can approximate complex nonlinear mappings.

- Loss:

$$\mathcal{L} = \frac{1}{N} \sum_i (y_i - \hat{y}_i)^2$$

- **Optimization:**

$$\theta \leftarrow \theta - \eta \nabla_{\theta} \mathcal{L} \quad \rightarrow \quad \theta = \{W^{(1)}, b^{(1)}, W^{(2)}, b^{(2)}\}$$

Backpropagation Concept

- **Goal:** Efficiently compute gradients of the loss with respect to *all* parameters.
- **Key principle:** *Chain Rule of Calculus.*

$$\frac{\partial \mathcal{L}}{\partial W^{(l)}} = \frac{\partial \mathcal{L}}{\partial a^{(l)}} \cdot \frac{\partial a^{(l)}}{\partial z^{(l)}} \cdot \frac{\partial z^{(l)}}{\partial W^{(l)}}$$

- **Example for Two Layers**

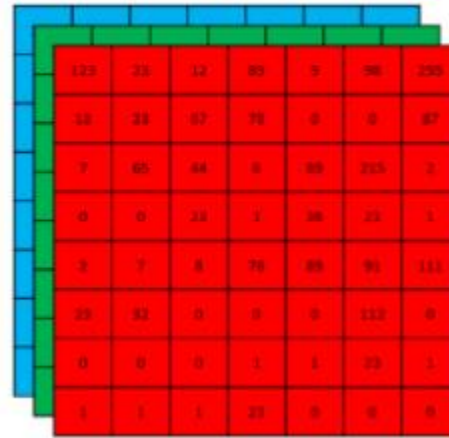
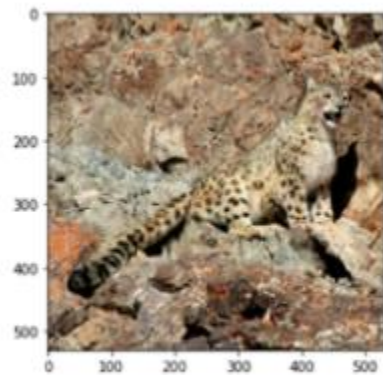
$$z^{(1)} = W^{(1)}x + b^{(1)}$$

$$a^{(1)} = \sigma(z^{(1)})$$

$$z^{(2)} = W^{(2)}a^{(1)} + b^{(2)}$$

$$\hat{y} = \sigma(z^{(2)})$$

Example: Binary Classification



$$X = [x^{(1)} \quad \dots \quad x^{(m)}] \in \mathbb{R}^{n \times m}$$

$$\hat{y} = P(y = 1|x), \quad 0 \leq \hat{y} \leq 1$$

Data Collection and Preprocessing

- **Dataset:** We would gather a dataset of images, some containing leopards (positive samples) and others with no leopards (negative samples). The dataset should include varied images to improve the model's robustness.
- **Labels:** Each image is labeled as 1 (Leopard) or 0 (No Leopard).
- **Preprocessing:** Images are resized, normalized, and possibly augmented (e.g., rotated, flipped, or cropped) to make the model more invariant to different conditions.

Model Architecture

- **Input Layer:** Each image, after preprocessing, is fed into the network as an array of pixel values.
- **Hidden Layers:** Convolutional layers are typically used for image classification tasks, as they are well-suited for extracting visual features such as edges, patterns, and textures. Deeper layers may capture more abstract features such as specific textures or parts of a leopard, like fur patterns or faces.
- **Output Layer:** The output layer for a binary classification problem usually consists of a single neuron with a Sigmoid activation function, which outputs a probability between 0 and 1.

Binary Classification

- Reminder: Binary Logistic Regression
- **The objective:** to train a classifier that can make a binary decision about the class of a new input observation.
- Sigmoid classifier is used to predict the label.
- Logistic regression solves this task by learning, from a training set, a **vector of weights** and a **bias** term.

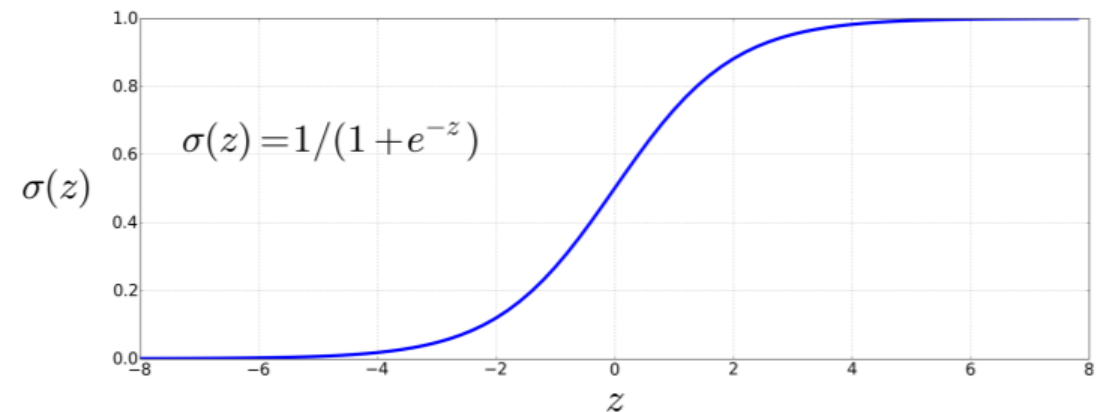
$$P(\hat{y} = 1|x)$$

Logistic Regression

$$P(\hat{y} = 1|x) \quad z = \left(\sum_{i=1}^n w_i x_i \right) + b$$
$$z = w \cdot x + b$$

- To calculate the probability, Let's pass z through the sigmoid function, $\sigma(z)$

$$\sigma(z) = \frac{1}{1 + e^{-z}} = \frac{1}{1 + \exp(-z)}$$



Logistic Regression

$$\begin{aligned} P(y = 1) &= \sigma(w \cdot x + b) \\ &= \frac{1}{1 + \exp(-(w \cdot x + b))} \end{aligned}$$

$$\begin{aligned} P(y = 0) &= 1 - \sigma(w \cdot x + b) \\ &= 1 - \frac{1}{1 + \exp(-(w \cdot x + b))} \\ &= \frac{\exp(-(w \cdot x + b))}{1 + \exp(-(w \cdot x + b))} \end{aligned}$$

- The algorithm gives us the probability $P(y = 1 | x)$. How do we make a decision?

$$\text{decision}(x) = \begin{cases} 1 & \text{if } P(y = 1 | x) > 0.5 \\ 0 & \text{otherwise} \end{cases}$$

Cross Entropy Loss Function

- The cross-entropy loss function is calculated after the **output layer** and is used to measure prediction accuracy during training.
- This loss is minimized through backpropagation, updating the network's weights to improve its classification performance.

Cross Entropy Loss Function

- Loss function should show, for an observation x , how close the classifier output ($\hat{y} = \sigma(w \cdot x + b)$) is to the correct output y (which is 0 or 1).

$L(\hat{y}, y)$ = How much $\hat{y}$ differs from the true y

- We use conditional maximum likelihood estimation: choose w, b that maximize the log probability of the true y labels in the training data given the observations x .

$$p(y|x) = \hat{y}^y (1 - \hat{y})^{1-y}$$

Cross Entropy Loss Function (Log Loss)

$$\begin{aligned}\log p(y|x) &= \log [\hat{y}^y (1 - \hat{y})^{1-y}] \\ &= y \log \hat{y} + (1 - y) \log (1 - \hat{y})\end{aligned}$$

- Let's turn this into loss function (to minimize)

$$L_{\text{CE}}(\hat{y}, y) = -\log p(y|x) = -[y \log \hat{y} + (1 - y) \log (1 - \hat{y})]$$

- Let's plugin the definition of $\hat{y}$:

$$L_{\text{CE}}(\hat{y}, y) = -[y \log \sigma(w \cdot x + b) + (1 - y) \log (1 - \sigma(w \cdot x + b))]$$

Other Loss Functions in Deep Learning

- MSE:

$$\mathcal{L}_{MSE} = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2$$

- MAE:

$$\mathcal{L}_{MAE} = \frac{1}{N} \sum_{i=1}^N |y_i - \hat{y}_i|$$

- Huber Loss (Smooth L1): Used For: Combining robustness (MAE) and stability (MSE).

- where δ is a threshold hyperparameter.

- **Interpretation**

- Quadratic near small errors (like MSE).
- Linear for large errors (like MAE).

$$\mathcal{L}_{Huber} = \begin{cases} \frac{1}{2} (y_i - \hat{y}_i)^2 & \text{if } |y_i - \hat{y}_i| < \delta \\ \delta (|y_i - \hat{y}_i| - \frac{1}{2} \delta) & \text{otherwise} \end{cases}$$

Other Loss Functions in Deep Learning

- **Cross-Entropy Loss (Log Loss):** Used For Classification tasks (binary or multi-class).

- **Binary Case**

$$\mathcal{L}_{BCE} = -\frac{1}{N} \sum_{i=1}^N [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)]$$

- **Multi-Class Case (Softmax + Cross Entropy)**

$$\mathcal{L}_{CE} = - \sum_{i=1}^N \sum_{c=1}^C y_{i,c} \log(\hat{y}_{i,c})$$

Or

$$\mathcal{L}_{CE} = -\frac{1}{N} \sum_{i=1}^N \sum_{c=1}^C y_{i,c} \log(\hat{y}_{i,c})$$

Other Loss Functions in Deep Learning

- Kullback-Leibler (KL) Divergence
- Used For Comparing two probability distributions (often in VAEs).
- Equation

$$D_{KL}(P\|Q) = \sum_i P(i) \log \frac{P(i)}{Q(i)}$$

$$\mathcal{L}_{KL} = D_{KL}(q_\phi(z|x)\|p(z))$$

- Measures how much predicted distribution Q diverges from true P .
- Asymmetric:

$$D_{KL}(P\|Q) \neq D_{KL}(Q\|P).$$

Other Loss Functions in Deep Learning

- **Categorical Hinge Loss**
- Used for Multi-class margin-based classification (e.g. SVMs).

$$\mathcal{L}_{hinge} = \max(0, 1 - y_i \cdot \hat{y}_i)$$

- For multi-class:

$$\mathcal{L}_{multi} = \max(0, \hat{y}_{wrong} - \hat{y}_{correct} + \Delta)$$

- Encourages correct class scores to exceed incorrect ones by at least margin .
- Used when outputs are scores, not probabilities.

Example of Categorical Hinge Loss

- The network's output layer does not use softmax here . it just produces raw scores (real numbers) for each class:

$$\hat{y} = [s_{cat}, s_{dog}, s_{bird}]$$

- Let's say for one image the model outputs:

$$\hat{y} = [2.0, 0.5, -1.0]$$

- and the **true label** is "Cat".

- So:

$$s_{\text{correct}} = s_{cat} = 2.0$$

- The **highest incorrect score** =

$$s_{\text{wrong}} = \max(0.5, -1.0) = 0.5$$

- If we set margin $\Delta = 1.0$,

$$\mathcal{L}_{\text{multi}} = \max(0, s_{\text{wrong}} - s_{\text{correct}} + \Delta)$$

$$\mathcal{L}_{\text{multi}} = \max(0, 0.5 - 2.0 + 1.0) = \max(0, -0.5) = 0$$

- **Loss = 0** , correct class score already exceeds all others by ≥ 1 .

Other Loss Functions in Deep Learning

- **Cosine Similarity Loss**
- Used for Sentence embeddings, recommendation systems, face recognition.

$$\mathcal{L}_{cosine} = 1 - \frac{\hat{y} \cdot y}{\|\hat{y}\|_2 \|y\|_2}$$

- Encourages model to produce embeddings with similar direction (angle), not necessarily magnitude.
- Common in contrastive or Siamese networks.

Gradient Descent

- **Objective:** to find the optimal weights that minimize the loss function.

$$\hat{\theta} = \underset{\theta}{\operatorname{argmin}} \frac{1}{m} \sum_{i=1}^m L_{\text{CE}}(f(x^{(i)}; \theta), y^{(i)})$$

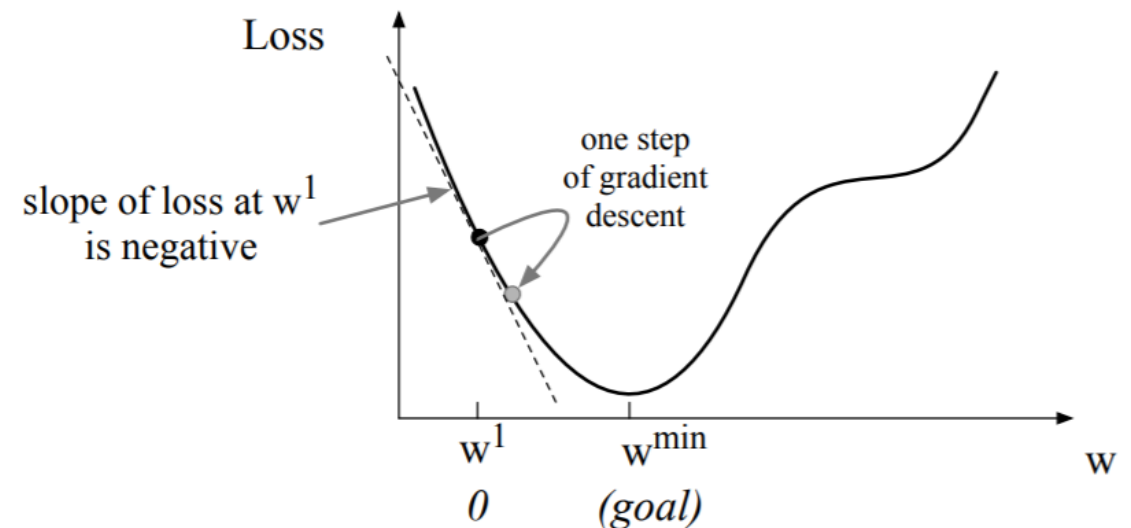
- In the case of logistic regression $\theta = w, b$

$$w^{t+1} = w^t - \eta \frac{d}{dw} L(f(x; w), y)$$

Gradient Descent

- In the case of logistic regression $\theta = w, b$
- The gradient descent algorithm finds the gradient of the loss function at the current point and moves in the opposite direction.
- The magnitude of move is the value of the slope $\frac{d}{dw}L(f(x; w), y)$ weighted by a learning rate η

$$w^{t+1} = w^t - \eta \frac{d}{dw}L(f(x; w), y)$$



Learning Rate

- **learning rate** determines how fast or slow the algorithm moves towards the optimal weights.

